

DS Assignment 5

Implement token ring based mutual exclusion algorithm.

// MutualServer.java

```
import java.io.*;
```

```
import java.net.*;
```

```
public class MutualServer {
```

```
    public static void main(String[] args) throws IOException {
```

```
        // Create a server socket on port 7000.
```

```
        // Both clients will connect to this server to send messages.
```

```
        ServerSocket ss = new ServerSocket(7000);
```

```
        System.out.println("Server started on port 7000");
```

```
        // Server runs continuously to accept multiple client connections.
```

```
        while (true) {
```

```
            // Accept a new client connection.
```

```
            Socket client = ss.accept();
```

```
            // Create a separate thread for each connected client.
```

```
            // This allows the server to receive messages from multiple clients.
```

```
            new Thread() -> {
```

```
                try {
```

```
                    // Used to read messages sent by the client.
```

```
                    BufferedReader in = new BufferedReader(
```

```
                        new InputStreamReader(client.getInputStream()));
```

```

        String msg;

        // Keep reading messages from the client until connection closes.
        while ((msg = in.readLine()) != null) {
            System.out.println("Message received: " + msg);
        }
    } catch (Exception e) {
        System.out.println("Client disconnected");
    }
    }).start();
}
}
}
}
}

```

// ClientOne.java

```

import java.io.*;
import java.net.*;

public class ClientOne {
    public static void main(String[] args) throws IOException {
        Socket serverSocket = new Socket("localhost", 7000);
        PrintWriter server = new PrintWriter(serverSocket.getOutputStream(), true);

        ServerSocket ss = new ServerSocket(7001);
        Socket ring = ss.accept();

        BufferedReader fromTwo = new BufferedReader(
            new InputStreamReader(ring.getInputStream()));
        PrintWriter toTwo = new PrintWriter(ring.getOutputStream(), true);
    }
}

```

```

BufferedReader console = new BufferedReader(
    new InputStreamReader(System.in));

String token = "Token";

while (true) {
    if (token != null && token.equalsIgnoreCase("Token")) {
        System.out.print("Send data? (yes/no): ");
        String choice = console.readLine();

        if (choice.equalsIgnoreCase("yes")) {
            System.out.print("Enter data: ");
            String data = console.readLine();
            server.println("ClientOne: " + data);
        }

        toTwo.println("Token");
        System.out.println("Token passed to ClientTwo");
    }

    System.out.println("Waiting for token...");
    token = fromTwo.readLine();
}
}
}

```

// ClientTwo.java

```

import java.io.*;

```

```
import java.net.*;

public class ClientTwo {
    public static void main(String[] args) throws IOException {
        Socket serverSocket = new Socket("localhost", 7000);
        PrintWriter server = new PrintWriter(serverSocket.getOutputStream(), true);

        Socket ring = new Socket("localhost", 7001);

        BufferedReader fromOne = new BufferedReader(
            new InputStreamReader(ring.getInputStream()));
        PrintWriter toOne = new PrintWriter(ring.getOutputStream(), true);

        BufferedReader console = new BufferedReader(
            new InputStreamReader(System.in));

        while (true) {
            System.out.println("Waiting for token...");
            String token = fromOne.readLine();

            if (token != null && token.equalsIgnoreCase("Token")) {
                System.out.print("Send data? (yes/no): ");
                String choice = console.readLine();

                if (choice.equalsIgnoreCase("yes")) {
                    System.out.print("Enter data: ");
                    String data = console.readLine();
                    server.println("ClientTwo: " + data);
                }
            }
        }
    }
}
```

```
        toOne.println("Token");  
        System.out.println("Token passed to ClientOne");  
    }  
}  
}  
}
```

Output-

Terminal 1:

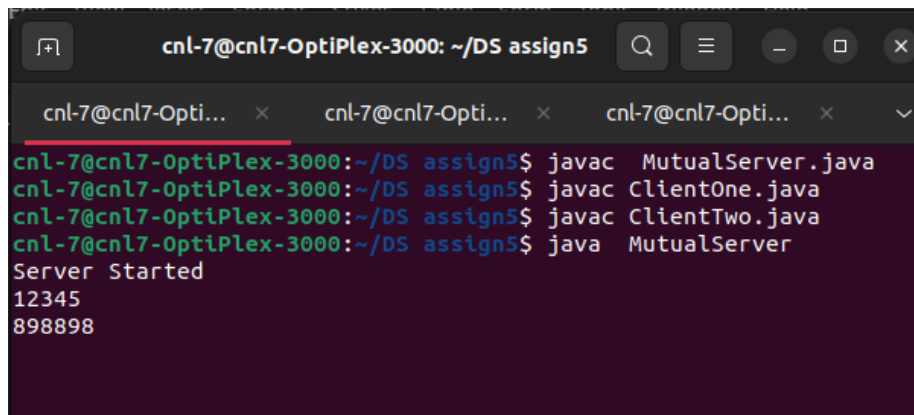
```
javac MutualServer.java ClientOne.java ClientTwo.java  
java MutualServer
```

Terminal 2:

```
java ClientOne
```

Terminal 3:

```
java ClientTwo
```



```
cnl-7@cnl7-OptiPlex-3000: ~/DS assign5  
cnl-7@cnl7-OptiPlex-3000:~/DS assign5$ javac MutualServer.java  
cnl-7@cnl7-OptiPlex-3000:~/DS assign5$ javac ClientOne.java  
cnl-7@cnl7-OptiPlex-3000:~/DS assign5$ javac ClientTwo.java  
cnl-7@cnl7-OptiPlex-3000:~/DS assign5$ java MutualServer  
Server Started  
12345  
898898
```

```
cnl-7@cnl7-OptiPlex-3000: ~/DS assign5
cnl-7@cnl7-OptiPlex-3000:~/DS assign5$ java ClientOne
Do you want to send some data
Enter Yes or No
yes
Enter the data
12345
Waiting for Token
Do you want to send some data
Enter Yes or No
```

```
cnl-7@cnl7-OptiPlex-3000: ~/DS assign5
cnl-7@cnl7-OptiPlex-3000:~/DS assign5$ java ClientTwo
Waiting for Token
Do you want to send some data
Enter Yes or No
yes
Enter the data
898898
Waiting for Token
```